TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer Engineering

A REPORT ON
WORLD UNIVERSITY RANKINGS

Submitted in partial fulfillment of the requirements for the degree

BACHELOR OF COMPUTER ENGINEERING

Submitted by

Anjana Silinchhe Shrestha	KCE080BCT004
Pradipta Joshi	KCE080BCT021
Prasant Rai	KCE080BCT024
Sushma Shrestha	KCE080BCT043

Under the Supervision of
Er. Bikesh Manandhar
Er. Suresh Shrestha
Er. Anish Baral

Khwopa College Of Engineering
Libali, Bhaktapur
March 2026

Certificate of Approval

The undersigned certify that the fifth semester project entitled **"World University Rankings"** submitted by Anjana Silinchhe Shrestha, Pradipta Joshi, Prasant Rai and Sushma Shrestha to the **Department of Computer Engineering** in partial fulfillment of requirement for the degree of **Bachelor of Engineering in Computer Engineering**. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled, bona fide and ready to undertake any commercial and industrial work related to their field of study and hence we recommend the award of Bachelor of Computer Engineering degree.

.....
Er. Bikesh Manandhar
Project Supervisor
Department of Computer
Engineering, KhCE

.....
Er. Suresh Shrestha
Project Supervisor
Department of Computer
Engineering, KhCE

.....
Er. Anish Baral
Subject Teacher
Department of Computer
Engineering, KhCE

.....
Er. Dinesh Gothe
Head of Department
Department of Computer
Engineering, KhCE

Copyright

The author has agreed that the library, Khwopa College of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for the extensive copying of this project report for scholarly purpose may be granted by supervisor who supervised the project work recorded here in or, in absence the Head of The Department where in the project report was done. It is understood that the recognition will be given to the author of the report and to Department of Computer Engineering, KhCE in any use of the material of this project report. Copying or publication or other use of this report for financial gain without approval of the department and author's written permission is prohibited. Request for the permission to copy or to make any other use of material in this report in whole or in part should be addressed to:

Head of Department
Department of Computer Engineering
Khwopa College of Engineering
Liwali,
Bhaktapur, Nepal

Acknowledgement

We take this opportunity to express our deepest and sincere gratitude to our HoD Er. Dinesh Gothe, for his insightful advice, motivating suggestions for this project, and his constant encouragement throughout our Bachelor's program.

We extend our heartfelt thanks to our project supervisors, Er. Bikesh Manandhar and Er. Suresh Shrestha, for their invaluable guidance, continuous support, and supervision throughout the development of this project. Their expertise and feedback were instrumental in shaping the direction of our work.

We are also grateful to our subject teacher, Er. Anish Baral, for his valuable suggestions and guidance that helped us refine our project.

Anjana Silinchhe Shrestha	KCE080BCT004
Pradipta Joshi	KCE080BCT021
Prasant Rai	KCE080BCT024
Sushma Shrestha	KCE080BCT043

Abstract

This project presents a comprehensive analysis of global university rankings data and an intelligent chatbot designed to provide users with a seamless and intuitive way to explore the rankings. Data was collected from three major ranking systems — QS World University Rankings 2026, ARWU 2025, and Times Higher Education (THE) 2026 — using custom web scrapers, followed by rigorous data cleaning and Exploratory Data Analysis (EDA). The chatbot allows users to query the rankings data using natural language, providing answers to a wide range of questions, from simple lookups to complex comparisons and aggregations. The system uses a dual-path query architecture combining Retrieval-Augmented Generation (RAG) with vector embeddings and a Text-to-SQL model, enabling accurate and contextually relevant responses. Additionally, a predictive machine learning model using XGBoost regression was developed on the THE 2026 dataset, employing K-Means clustering to group universities into five structural archetypes and achieving an R^2 of 0.79, demonstrating that structural characteristics such as student count, faculty ratio, and country can predict a university's overall ranking score with high accuracy. The project aims to make university ranking data more accessible and insightful for students, researchers, and anyone interested in higher education.

Keywords: *World University Rankings, Web Scrapping, Data Cleaning, EDA, Model Training, Chatbot, RAG, Text-to-SQL, Vector Embeddings, Natural Language Processing*

Contents

Certificate of Approval i
Copyright i
Acknowledgement ii
Abstract iii
Contents v
List of Figures viii
List of Symbols and Abbreviation ix

1 Introduction **1**
1.1 Background Introduction 1
1.2 Motivation 1
1.3 Problem Definition 1
1.4 Goals and Objectives 2
1.5 Scope and Applications 2
1.6 Report Organization 2
1.6.1 Introduction 2
1.6.2 Literature Review 2
1.6.3 Feasibility Study 2
1.6.4 Methodology 3
1.6.5 Requirement Analysis 3
1.6.6 System Design and Architecture 3
1.6.7 Implementation and Testing 3
1.6.8 Results and Discussion 3
1.6.9 Conclusion and Future Enhancements 3

2 Literature Review **4**
2.1 Natural Language Processing and Information Retrieval 4
2.2 Web Scraping and Data Collection 4
2.3 Data Analysis with Python 4
2.4 Chatbot Systems and User Interfaces 5
2.5 University Ranking Systems 5

3 Feasibility Study **6**
3.1 Technical Feasibility 6
3.2 Operational Feasibility 6
3.3 Economic Feasibility 7
3.4 Scheduling Feasibility 7

4 Methodology **8**
4.1 Project Workflow Overview . . . 8
4.1.1 Phase 1: Data Collection and Exploratory Data Analysis . . 8
4.1.2 Phase 2: Chatbot and Machine Learning Model Development 9
4.2 Data Acquisition . . . 9
4.2.1 QS World University Rankings: Playwright Scraper . . . 9
4.2.2 QS Rankings — REST API Approach . . . 10
4.2.3 ARWU Rankings — Shanghai Rankings API . . . 10
4.2.4 THE World University Rankings — JSON Endpoint . . . 10
4.3 Data Processing and Storage . . . 10
4.4 Vector Embeddings and ChromaDB . . . 10
4.5 Chatbot Engine and Query Routing . . . 11
4.6 Machine Learning Model . . . 11
4.6.1 Model Purpose . . . 12
4.6.2 Data Cleaning . . . 12
4.6.3 Feature Engineering . . . 12
4.6.4 Unsupervised Clustering . . . 12
4.6.5 Preprocessing Pipeline . . . 13
4.6.6 Model Comparison . . . 13
4.6.7 Evaluation Metrics . . . 14
4.6.8 Feature Importance . . . 14
4.6.9 Future Scope . . . 15
4.6.10 Complete Feature List . . . 15
4.7 User Interface . . . 15

5 Requirement Analysis **16**
5.1 Software Requirements . . . 16
5.2 Hardware Requirements . . . 17
5.3 Functional Requirements . . . 17
5.4 Non-Functional Requirements . . . 18

6 System Design and Architecture **19**
6.1 System Architecture . . . 19
6.2 System Context . . . 20
6.3 Use Case Diagram . . . 21
6.4 Sequence Diagram . . . 21
6.5 Database Schema . . . 22
6.6 ChromaDB Vector Store Design . . . 22
6.7 Query Routing Logic . . . 23

7 Implementation and Testing **24**
7.1 Implementation Details . . . 24
7.1.1 User Interface Implementation . . . 24
7.1.2 Scraper (scraper.py) . . . 25
7.1.3 Data Pipeline (pipeline.py) . . . 25
7.1.4 Embedder (embedder.py) . . . 26
7.1.5 Chatbot Query Router (chatbot.py) . . . 26
7.1.6 Streamlit Application (app.py) . . . 27
7.1.7 Machine Learning Notebook . . . 27

7.2 Testing . . . 27
7.2.1 Scraper Validation . . . 28
7.2.2 Pipeline Validation . . . 28
7.2.3 Chatbot Path Testing . . . 28
7.2.4 End-to-End Testing . . . 28

8 Results and Discussion **29**
8.1 Chatbot Performance . . . 29
8.1.1 Profile Path — Sample Queries . . . 29
8.1.2 SQL Path — Sample Queries . . . 29
8.1.3 RAG Path — Sample Queries . . . 30
8.2 Data Analysis and Exploratory Data Analysis (EDA) . . . 30
8.2.1 Geographic Distribution of Universities . . . 30
8.2.2 Missing Values Analysis . . . 30
8.2.3 QS Rankings EDA . . . 31
8.2.4 ARWU Rankings EDA . . . 31
8.2.5 THE EDA and Machine Learning Results . . . 31
8.3 Discussion . . . 34

9 Conclusion and Future Enhancements **35**
9.1 Conclusion . . . 35
9.2 Future Enhancements . . . 35
Appendix . . . 37
9.3 Glossary of Terms . . . 37
9.4 Technologies Used . . . 38
Bibliography . . . 39

List of Figures

6.1 System Architecture Diagram 19
6.2 System Context Diagram 20
6.3 Use Case Diagram 21
6.4 Sequence Diagram for Query Processing 22

7.1 Chatbot Landing Page 24
7.2 Basic Greeting Interaction 25
7.3 Chatbot Processing a University Query 26

8.1 Geographic Distribution of Universities by Country 30
8.2 Missing Values Heatmap 31
8.3 Correlation Heatmap of QS Scoring Indicators 32
8.4 Number of Universities per Country in QS Rankings 32
8.5 Top 20 Universities by Overall Score 33
8.6 Data Preprocessing Pipeline 33
8.7 Model Performance Comparison 33
8.8 XGBoost Feature Importance 34

List of Abbreviation

Abbreviations	**Meaning**
API	Application Programming Interface
ARWU	Academic Ranking of World Universities
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
NLP	Natural Language Processing
NLU	Natural Language Understanding
QS	Quacquarelli Symonds
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
THE	Times Higher Education
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modelling Language

Chapter 1

Introduction

1.1 Background Introduction

University rankings have become a crucial tool for students, academics, and institutions worldwide [1]. They provide a comparative view of universities based on various indicators such as academic reputation, employer reputation, research citations, and more. Major ranking systems like the QS World University Rankings, Times Higher Education (THE) World University Rankings, and the Academic Ranking of World Universities (ARWU) offer comprehensive data, but navigating and comparing information across these different platforms can be a challenging and time-consuming task [2].

1.2 Motivation

The motivation for this project stems from the need to simplify the process of accessing and comparing university rankings. With vast amounts of data spread across multiple sources, prospective students and researchers often face information overload. A centralized and user-friendly interface can significantly enhance the user's ability to query specific information and gain insights from the rankings. A chatbot presents an intuitive and interactive way to address this need, providing instant answers to natural language questions.

1.3 Problem Definition

The primary problem this project aims to solve is the difficulty in efficiently querying and comparing data from various university ranking systems [1]. Users often have to manually visit multiple websites, understand different methodologies, and compile data to make informed decisions. This process is inefficient and prone to errors. The project addresses this by developing a chatbot that serves as a single point of access to a consolidated database of university rankings.

1.4 Goals and Objectives

The main goal of this project is to develop a chatbot that can intelligently answer user queries about world university rankings. The specific objectives are as follows:

- To scrape and collect university ranking data from multiple sources, including QS, THE, and ARWU.
- To clean, preprocess, and store the collected data in a structured format.
- To develop a natural language understanding (NLU) module to interpret user queries.
- To build a chatbot that can retrieve and present relevant information from the database in response to user queries.
- To create a user-friendly interface for interacting with the chatbot.

1.5 Scope and Applications

The scope of this project is to create a chatbot focused on providing information from the QS, THE, and ARWU world university rankings. The chatbot will be able to answer a wide range of questions related to university ranks, scores, and locations. The applications of this project are diverse:

- Prospective students can use it to find and compare universities based on their preferences.
- University administrators can use it for competitive analysis and benchmarking.
- Educational consultants can use it to provide quick and accurate information to their clients.

1.6 Report Organization

1.6.1 Introduction

This chapter provides an overview of the project, including the background, motivation, problem statement, goals, and scope.

1.6.2 Literature Review

This chapter will review existing research and projects related to chatbots, natural language processing, and university ranking systems.

1.6.3 Feasibility Study

This section will analyze the technical, economic, and operational feasibility of the proposed system.

1.6.4 Methodology

This chapter will detail the methodology used for data scraping, data preprocessing, chatbot development, and system architecture.

1.6.5 Requirement Analysis

This section will outline the functional and non-functional requirements of the system.

1.6.6 System Design and Architecture

This chapter will describe the overall design and architecture of the chatbot system, including the database schema and the interaction between different components.

1.6.7 Implementation and Testing

This chapter will discuss the implementation details of the project and the testing strategies used to evaluate the system.

1.6.8 Results and Discussion

This section will present the results of the project and discuss the findings.

1.6.9 Conclusion and Future Enhancements

This chapter will summarize the project and suggest potential areas for future work and improvements.

Chapter 2

Literature Review

2.1 Natural Language Processing and Information Retrieval

Natural Language Processing (NLP) enables computers to understand and process human language [3]. Key NLP tasks include tokenization, named entity recognition, and intent classification—all essential for parsing user queries in a chatbot system. Modern NLP libraries such as spaCy provide efficient pipelines for these tasks [4].

Information Retrieval (IR) focuses on obtaining relevant information from large document collections [2]. Question Answering (QA) systems combine IR with NLP to provide direct answers to natural language questions. This project implements a QA system that retrieves university ranking information through both structured SQL queries and semantic vector search.

2.2 Web Scraping and Data Collection

Web scraping is the automated extraction of data from websites [5]. Libraries such as BeautifulSoup parse HTML to extract structured data, while tools like Playwright handle JavaScript-rendered content. For this project, data was collected from QS, THE, and ARWU ranking websites using both REST API calls and headless browser automation.

2.3 Data Analysis with Python

Python has become the dominant language for data analysis due to its rich ecosystem of libraries [6]. Pandas provides data structures and operations for manipulating numerical tables and time series, which is essential for cleaning and processing university ranking data. The data science workflow—from data collection through cleaning, analysis, and visualization—is well-supported by Python's scientific computing stack [1].

2.4 Chatbot Systems and User Interfaces

Conversational interfaces have evolved from simple rule-based systems to intelligent agents powered by large language models. Modern chatbots combine multiple technologies: natural language understanding, database querying, and response generation. Web frameworks like Streamlit enable rapid development of interactive applications, making it feasible to deploy chatbot interfaces with minimal boilerplate code [7].

2.5 University Ranking Systems

Three major global ranking systems provide comprehensive university performance data:

- **QS World University Rankings:** Based on academic reputation, employer reputation, faculty-student ratio, citations, and international diversity. The 2026 edition covers 1,504 institutions.
- **Times Higher Education (THE):** Uses 13 indicators across teaching, research, citations, international outlook, and industry income. The 2026 dataset includes 3,119 universities.
- **ARWU (Shanghai Rankings):** Focuses on research performance, including Nobel Prizes, highly cited researchers, and publications in Nature and Science.

Consolidating data from these sources enables multi-faceted analysis and comparison of university performance across different methodologies.

Chapter 3

Feasibility Study

3.1 Technical Feasibility

The project is technically feasible as it relies on well-established and accessible technologies.

- **Data Collection:** The QS website renders its ranking table via JavaScript, requiring a headless browser. Playwright with the `playwright-stealth` plugin is a well-documented open-source tool that handles this. THE and ARWU data are available via direct JSON REST endpoints.
- **Data Storage:** SQLite is bundled with Python and requires no separate server, making it straightforward to create, populate, and query. ChromaDB is an open-source vector database that persists to disk.
- **Chatbot Logic:** LLM APIs (Groq free tier, OpenAI) handle query classification, SQL generation, and response synthesis. Sentence Transformers provide embedding inference locally without GPU requirements. All required libraries are open-source.
- **User Interface:** Streamlit is a Python-native framework that converts Python scripts into interactive web applications with minimal boilerplate.

All the required software is open-source, and there is extensive documentation and community support available.

3.2 Operational Feasibility

The system is designed to be straightforward to operate and maintain.

- **Data Updates:** The ranking data needs to be updated periodically (e.g., annually) when new rankings are released. The scraping scripts can be re-run to refresh the database. This process can be automated.
- **User Interaction:** The chatbot will provide a simple, text-based interface that is intuitive for users. No special training will be required to use the system.

- **Maintenance:** The primary maintenance task will be to update the web scrapers if the source websites change their HTML structure. This is a manageable task for the development team.

3.3 Economic Feasibility

The project is highly economically feasible.

- **Software Costs:** All core libraries (Playwright, ChromaDB, Sentence Transformers, pandas, scikit-learn, XGBoost, Streamlit) are open-source and free. LLM API access is provided via the Groq free tier (Llama 3), which offers sufficient request quotas for development.
- **Hardware Costs:** The embedding model runs on CPU without a GPU, making standard development-grade laptops sufficient. No cloud compute or specialised hardware is required.
- **Development Costs:** The main cost is the time and effort of the project team. As this is an academic project, these costs are part of the learning process.

Given the reliance on free and open-source software and standard hardware, the project has a very low financial barrier.

3.4 Scheduling Feasibility

The project can be completed within a reasonable timeframe by breaking it down into manageable phases. A Gantt chart outlining the schedule for each phase is provided in the report. The key phases include:

- Project Planning and Requirement Analysis
- Data Scraping and Database Design
- Data Cleaning and Preprocessing
- Chatbot Core Logic and NLP Implementation
- User Interface Development
- System Integration and Testing
- Documentation and Final Report

With proper time management and allocation of tasks, the project is schedule-feasible.

Chapter 4

Methodology

4.1 Project Workflow Overview

The project was executed in two distinct phases, with team members contributing collaboratively at each stage.

4.1.1 Phase 1: Data Collection and Exploratory Data Analysis

In the initial phase, each team member independently developed web scrapers to collect university ranking data from different sources:

- **QS World University Rankings:** Scrapers were developed using Playwright with stealth plugins and direct REST API calls to extract the QS 2026 rankings comprising 1,504 universities.
- **ARWU Rankings:** A scraper was built targeting the Shanghai Rankings REST API to collect ARWU 2025 data.
- **THE World University Rankings:** Data was extracted from the Times Higher Education JSON endpoint, yielding 3,119 universities.

Following data collection, each member performed Exploratory Data Analysis (EDA) on their respective datasets. This included:

- Data quality assessment and missing value analysis
- Statistical summaries and distribution analysis
- Correlation analysis between ranking indicators
- Geographic distribution of universities
- Identification of patterns and anomalies in ranking data

The EDA findings informed the subsequent data cleaning and preprocessing pipeline, as well as the design of the chatbot's query capabilities.

4.1.2 Phase 2: Chatbot and Machine Learning Model Development

Building on the cleaned datasets, the second phase introduced two major components:

- **University Rankings Chatbot:** A conversational AI system was developed to answer natural language queries about university rankings. The chatbot integrates multiple technologies: an LLM-based query router, SQLite database for structured queries, and ChromaDB vector store for semantic search via RAG (Retrieval-Augmented Generation).
- **Predictive Machine Learning Model:** An XGBoost regression model was trained on THE 2026 data to predict a university's overall ranking score based on structural characteristics (student count, student-to-staff ratio, international student percentage, and country).

The following sections detail the technical methodology for each component.

4.2 Data Acquisition

4.2.1 QS World University Rankings: Playwright Scraper

The QS website renders its ranking table through JavaScript, making a standard `requests` + `BeautifulSoup` approach insufficient. The project therefore uses **Playwright** (headless Chromium) augmented with `playwright_stealth` and a rotating User-Agent pool to avoid bot detection. The scraper (`scraper.py`) implements a three-strategy fallback hierarchy:

1. **Strategy 0 — Direct REST API:** Extracts the ranking node ID (`nid`) from the page's embedded `drupalSettings` JSON object and calls the QS REST endpoint directly. This path retrieves all 1 504 universities in paginated JSON responses and is the fastest approach.
2. **Strategy 1 — Network Interception:** Registers a Playwright request-interceptor that captures every outgoing `fetch()` call as the page loads. If the rankings JSON payload is seen in the intercepted traffic it is extracted without any additional HTTP call.
3. **Strategy 2 — DOM Link Scan:** Falls back to parsing all university anchor tags from the rendered DOM when the API node ID cannot be found.

Once the main list (Layer A) is collected, a second scraping pass (Layer B) visits the individual profile page of each of the top 200 universities to extract: QS Lens scores (Research, Learning, Employability, Sustainability, Global Engagement), a prose description, key facts (institution type, founding year, total students), and student review snippets. All results are persisted to `data/raw/` as `rankings_main.csv` and `rankings_detail.csv`.

4.2.2 QS Rankings — REST API Approach

Scrapers call the `topuniversities.com` rankings endpoint directly with `requests`, sending paginated `GET` requests. The JSON response nests per-university scores under a `scores` dictionary. `BeautifulSoup` strips residual HTML tags from the university name field. Pagination terminates when a page returns fewer records than requested.

4.2.3 ARWU Rankings — Shanghai Rankings API

The ARWU scraper queries the Shanghai Rankings REST endpoint with `version=2025`. Indicator metadata is read from the JSON response and mapped to column names dynamically. The result is written to CSV.

4.2.4 THE World University Rankings — JSON Endpoint

THE data is downloaded from a JSON endpoint on `timeshighereducation.com`. The response contains one record per university with nested per-indicator scores. This dataset (3 119 universities) is stored as `the_world_rankings_2026.csv` and is used for the ML experiments (Section 4.6).

4.3 Data Processing and Storage

The `pipeline.py` module consumes the raw CSVs and performs the following operations in sequence:

1. **Score cleaning:** A `clean_score()` function converts QS-specific missing-value tokens (em-dash `"---"`, empty strings) to `pd.NA` and casts remaining values to `float`.
2. **Rank cleaning:** A `clean_rank()` function handles prefix formats (`"=42"`) and range formats (`"501-510"`) by extracting the lower bound as an integer.
3. **Country-to-continent mapping:** Each university is assigned a continent label using a curated dictionary, enabling continent-level aggregation queries.
4. **Merging:** The main and detail DataFrames are joined with `pd.merge()` on the university identifier to produce `rankings_merged.csv`.
5. **SQLite ingestion:** Using Python's `sqlite3` module, the pipeline creates two tables: `universities` (core scores) and `university_details` (QS Lens scores, descriptions). A `v_full_rankings` SQL view joins them.

4.4 Vector Embeddings and ChromaDB

The `embedder.py` module converts structured university records into semantic vector representations for the RAG retrieval path:

1. A rich prose document is assembled for each university by concatenating name, country, rank, scores, description, and review snippets.

2. **Sentence Transformers** (`all-MiniLM-L6-v2` model) encodes each document into a dense embedding vector.

3. Embeddings are upserted into a **ChromaDB** persistent collection in batches of 100 to prevent memory overload. Rich metadata (rank, country, continent, individual scores) is stored alongside each embedding to enable filtered retrieval.

4. The vector store is persisted to `data/chroma_db/` so it does not need to be rebuilt on every application launch.

4.5 Chatbot Engine and Query Routing

The chatbot (`chatbot.py`) implements an intelligent three-path routing strategy. On receiving a user message, the LLM first classifies the query type and then dispatches to the appropriate sub-engine:

- **Profile Path:** If the query mentions a specific university and requests descriptive information (e.g., *"Tell me about MIT"*), the router executes a parameterised `SELECT` against the `v_full_rankings` view and returns the full university profile including description, QS Lens scores, key facts, and student reviews.

- **SQL Path:** If the query requests aggregation, sorting, or mathematical operations (e.g., *"Average overall score of universities in Germany"* or *"Top 10 in Asia"*), the LLM generates a SQL statement against the known schema. The SQL is executed against the SQLite database and the raw results are passed back to the LLM to formulate a natural-language answer.

- **RAG Path:** For open-ended, comparative, or qualitative questions (e.g., *"Which UK university has the highest sustainability score?"*), the query is encoded with Sentence Transformers and the $k = 5$ most semantically similar university documents are retrieved from ChromaDB via cosine similarity. These retrieved chunks are injected into the LLM prompt as context, following the Retrieval-Augmented Generation (RAG) pattern.

The routing classification prompt instructs the LLM to return one of three labels—`profile`, `sql`, or `rag`—along with an extracted entity when applicable. Both Groq (free tier, Llama 3) and OpenAI API keys are supported; the system selects whichever provider is configured at startup. Conversation history is maintained and prepended to each prompt to support multi-turn dialogue.

4.6 Machine Learning Model

An independent predictive model was built on the THE 2026 dataset (`mini.ipynb`) to analyse how well the structural characteristics of a university—without using THE's own scoring pillars as input—can predict its overall ranking score.

4.6.1 Model Purpose

The model predicts a university's **overall score** (`scores_overall`) from the THE 2026 rankings using only demographic and geographic statistics: number of students, student-to-staff ratio, percentage of international students, and country. It does **not** use the five THE pillar scores (Teaching, Research, Citations, Industry Income, International Outlook) as input features, which means it learns to predict ranking position from structural characteristics rather than from the scoring pillars directly.

This approach is useful for answering questions such as: *"Based on a university's size, faculty ratio, and country—what overall score should it expect?"* It also enables identification of universities that are **over- or under-performing** relative to their structural profile.

4.6.2 Data Cleaning

The raw THE data contains numbers in non-standard formats. A cleaning function handles:

- Rank ranges (e.g., `"501-600"` → midpoint `550.5`)
- Student counts with commas and `+` suffix (e.g., `"10,000+"` → `10000.0`)
- Missing values (`"n/a"` or empty strings → `NaN`)
- Percentage strings (e.g., `"45.3%"` → `45.3`)

Rows where `scores_overall` is missing are dropped entirely (these are unranked "Reporter" universities that submit data but receive no score). This reduces the usable dataset from 3 119 to approximately 1 900+ scored universities.

4.6.3 Feature Engineering

Two engineered features supplement the raw input variables (student count, student-to-staff ratio, international student percentage, country):

- **location_strength:** The mean `scores_overall` of all universities in the same country, computed via `groupby('location').mean()`. This target encoding captures national research culture, government funding levels, and historical reputation as a single numeric signal—without requiring 80+ one-hot dummy variables for country.
- **intl_ratio:** The international student percentage rescaled from $[0, 100]$ to $[0, 1]$ to align with other features after `StandardScaler` normalisation.

4.6.4 Unsupervised Clustering

Before regression, universities are grouped into **five structural archetypes** with K-Means clustering on total students, student-to-staff ratio, and `intl_ratio`.

Rationale: Universities are not a homogeneous population. A small elite research university (e.g., MIT: ∼11 000 students, very high international ratio) and

a large national university (e.g., State University of Mexico: 300 000 students, very low international ratio) are structurally different. A single linear model would struggle to fit both simultaneously. By first clustering universities into 5 archetypes, the model can learn separate scoring patterns per group.

The five archetypes roughly correspond to:

1. **Elite research institutions**—small, highly international, high staff ratios
2. **Large national universities**—many students, domestic focus
3. **Mid-size international**—moderate size, high international ratio
4. **Regional universities**—smaller, low international diversity
5. **Specialist institutions**—unusual ratio profiles (e.g., medical or technical schools)

A `Pipeline` with median imputation and standard scaling precedes K-Means with $k = 5$, `random_state=42`, and `n_init=10`. The cluster label is cast to `str` immediately after assignment so that downstream `OneHotEncoder` treats it as a categorical variable.

4.6.5 Preprocessing Pipeline

A `ColumnTransformer` combines numeric and categorical preprocessing:

- **Numeric features:** Median imputation followed by `StandardScaler`. Scaling ensures features with different ranges contribute proportionally.
- **Categorical feature:** `OneHotEncoder` converts the 5 cluster labels into binary columns.

4.6.6 Model Comparison

Three models are trained and evaluated on the same 80/20 train-test split:

- **Baseline (Shallow RF):** `RandomForestRegressor(max_depth=2)`—a deliberately weak model to confirm the features carry predictive signal.
- **Random Forest:** `RandomForestRegressor(n_estimators=100)`—builds many decision trees independently and averages their predictions. This reduces overfitting compared to a single deep tree and handles non-linear relationships well.
- **XGBoost:** `XGBRegressor` with 300 estimators, learning rate 0.05, max depth 6—builds trees sequentially, correcting residual errors.

The key hyperparameters for XGBoost:

- `n_estimators=300`: More trees = finer error correction; stopped at 300 to avoid diminishing returns.

- `learning_rate=0.05`: Low learning rate forces the model to learn slowly and carefully—reduces overfitting.
- `max_depth=6`: Allows each tree to model moderately complex interactions without memorising training data.
- `subsample=0.8`: Stochastic sampling introduces noise that acts as regularisation.

Each model is wrapped in a `sklearn.pipeline.Pipeline` containing the preprocessor so that imputation and scaling are part of the model object—this prevents data leakage during cross-validation.

4.6.7 Evaluation Metrics

Two metrics are used for model evaluation:

- **R^2 (coefficient of determination):** Fraction of variance in `scores_overall` explained by the model. $R^2 = 1.0$ is perfect; $R^2 = 0.0$ means the model is no better than predicting the mean.
- **MAE (Mean Absolute Error):** Average absolute difference between predicted and actual score in the same units as `scores_overall` (0–100 scale).

5-Fold Cross-Validation provides a more robust performance estimate: the training set is split into 5 equal folds; for each fold, 4 folds train and 1 validates. The R^2 is computed for each validation set and averaged. A small standard deviation (e.g., ± 0.02) indicates stable generalisation; a large one (e.g., ± 0.15) suggests sensitivity to training data.

4.6.8 Feature Importance

XGBoost exposes a `feature_importances_` array after training, representing how much each feature contributed to reducing prediction error across all 300 trees.

Key Findings:

- **Location Strength** is the dominant predictor—geography concentrates research funding, government investment, and institutional prestige within specific countries.
- **Student-Staff Ratio** is the second-strongest driver—smaller class sizes correlate directly with higher scores. This reflects both teaching quality and the institutional wealth required to maintain low ratios at scale.
- **Archetype cluster label** adds meaningful signal—certain clusters (e.g., highly international elite universities) show distinct score distributions that a model without clustering would conflate.
- **International Ratio** acts as a "kingmaker" metric—universities that attract global students tend to punch above their weight in overall rankings compared to domestically-focused institutions of similar size.

- **XGBoost outperformed Random Forest** because the relationship between features and `scores_overall` is non-linear and involves interaction effects.

4.6.9 Future Scope

Including `scores_citations` or `scores_industry_income` as features would likely raise R^2 further since the current model relies only on demographic/geographic features and intentionally excludes the THE pillar scores.

4.6.10 Complete Feature List

Table 4.1 summarises all features fed to the model.

Feature	Type	Source	Notes
`stats_number_students`	Numeric	Raw	Total enrolled students
`stats_student_staff_ratio`	Numeric	Raw	Students per staff
`intl_ratio`	Numeric	Engineered	Intl. student % / 100
`location_strength`	Numeric	Target enc.	Mean score by country
`univ_archetype`	Categorical	K-Means	One of 5 archetypes

Table 4.1: Features used for predicting overall score

Target variable: `scores_overall`—THE's composite ranking score (0–100 scale).

4.7 User Interface

The Streamlit application (`app.py`) presents three panels:

- **Chat Tab:** A persistent conversation window with message history, a text input field, and a sidebar displaying quick statistics (total universities indexed, countries represented, and top-ranked university).
- **Data Explorer Tab:** An interactive table view of `rankings_merged.csv` with column-level filters and sliders.
- **About Tab:** Architecture overview and API key configuration status.

The application loads the ChromaDB collection and SQLite database once at startup and retains them in memory using Streamlit's `@st.cache_resource` decorator to eliminate repeated disk I/O on every request.

Chapter 5

Requirement Analysis

5.1 Software Requirements

The development of the World University Rankings Chatbot requires the following software components:

- **Operating System:** Windows, macOS, or Linux.
- **Programming Language:** Python 3.8 or higher.
- **Python Libraries:**
  - `streamlit`: For creating the web-based conversational user interface.
  - `playwright` and `playwright-stealth`: For headless browser automation and anti-bot-detection scraping.
  - `requests` and `BeautifulSoup4`: For direct REST API calls and HTML tag stripping.
  - `pandas`: For data manipulation, cleaning, merging, and CSV I/O.
  - `sqlite3` (stdlib): For creating and querying the SQLite database.
  - `chromadb`: Vector database for storing semantic embeddings.
  - `sentence-transformers`: For generating dense text embeddings.
  - `groq` / `openai`: LLM API clients for query routing and response synthesis.
  - `scikit-learn`: For preprocessing pipelines, K-Means clustering, and Random Forest regression.
  - `xgboost`: For gradient-boosted regression on THE rankings data.
  - `python-dotenv`: For loading API keys from a `.env` file at runtime.
- **Development Environment:** An IDE such as Visual Studio Code.
- **Version Control:** Git for source code management.
- **Documentation:** A LaTeX editor like Texmaker for report writing.

5.2 Hardware Requirements

The hardware requirements for developing and running the system are minimal:

- A standard desktop or laptop computer.
- Minimum 8 GB of RAM.
- An internet connection for web scraping and accessing online resources.
- No specialized hardware such as high-end GPUs is required.

5.3 Functional Requirements

The system must perform the following functions:

- **FR1: User Query Input:** The system shall provide a text input field for the user to enter their questions in natural language.
- **FR2: Query Processing:** The system shall parse the user's query to identify the main intent and key entities (e.g., university name, ranking system, year, location).
- **FR3: Data Retrieval:** The system shall retrieve the relevant information from the stored database based on the parsed query.
- **FR4: Response Generation:** The system shall present the retrieved information to the user in a clear and understandable format.
- **FR5: Query Routing:** The system shall automatically classify each user query as one of three types and dispatch it accordingly:
  - **Profile queries:** Return the full university profile including description, QS Lens scores, key facts, and student reviews.
  - **Aggregation/SQL queries:** Generate and execute SQL against the SQLite database to answer questions involving counts, averages, rankings, and comparisons.
  - **Semantic/RAG queries:** Retrieve the most relevant university documents from the ChromaDB vector store and synthesise a natural-language answer via LLM.
- **FR6: University Profile Lookup:** The system shall be able to return detailed university profiles including description, key facts (founding year, student count, institution type), QS Lens scores (Research, Learning, Employability, Sustainability, Global Engagement), and student review excerpts.
- **FR7: Multi-turn Conversation:** The system shall maintain conversation history and use prior context when generating subsequent responses.
- **FR8: Error Handling:** The system shall provide a helpful message if a query cannot be understood, if generated SQL is invalid, or if the requested information is not available in the database.

5.4 Non-Functional Requirements

These requirements define the quality attributes of the system:

- **NFR1: Performance:** The system should process a user query and return a response within 3-5 seconds.
- **NFR2: Usability:** The user interface should be simple, intuitive, and easy to use for non-technical users.
- **NFR3: Reliability:** The system should be available and function correctly under normal usage. The data provided should be accurate based on the last scrape.
- **NFR4: Maintainability:** The code should be well-structured, commented, and modular to facilitate future updates and bug fixes, especially for the web scraping component.
- **NFR5: Scalability:** The system should be designed to handle an increase in the amount of data (e.g., adding more ranking years or systems) without a significant degradation in performance.

Chapter 6

System Design and Architecture

6.1 System Architecture

The architecture of the World University Rankings Chatbot is designed to be modular and scalable. It consists of four main layers: the Data Collection Layer, the Data Processing Layer, the Query Engine Layer, and the Presentation Layer. Figure 6.1 illustrates the overall system architecture.

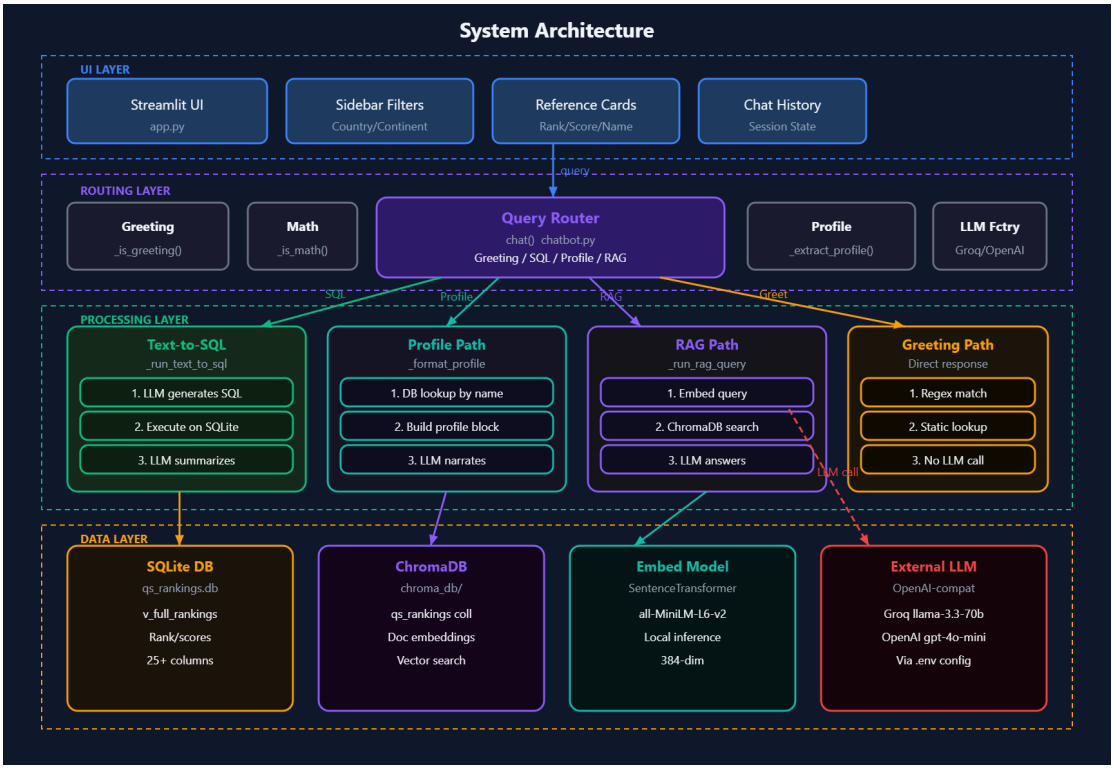

Figure 6.1: System Architecture Diagram

- **Data Collection Layer:** Playwright stealth scrapers and REST API clients gather raw ranking data from QS, THE, and ARWU into CSV files. Each data source uses the approach best suited to its structure (browser automation for the JavaScript-heavy QS website; direct JSON endpoints for THE and ARWU).

- **Data Processing Layer:** The `pipeline.py` script cleans and normalises the raw CSVs, persists them as structured SQLite tables and a SQL view, and the `embedder.py` script builds the ChromaDB vector store with Sentence Transformer embeddings.
- **Query Engine Layer:** The `chatbot.py` module implements a three-path router. It uses an LLM call to classify each query as `profile`, `sql`, or `rag`, then dispatches to the appropriate handler.
- **Presentation Layer:** The `app.py` Streamlit application provides a chat window with conversation history, a quick-statistics sidebar, and a data explorer tab.

6.2 System Context

The World University Rankings Chatbot operates as a standalone conversational system that interacts with users through a web-based interface. The system context diagram in Figure 6.2 illustrates the boundaries of the system and its interactions with external entities.

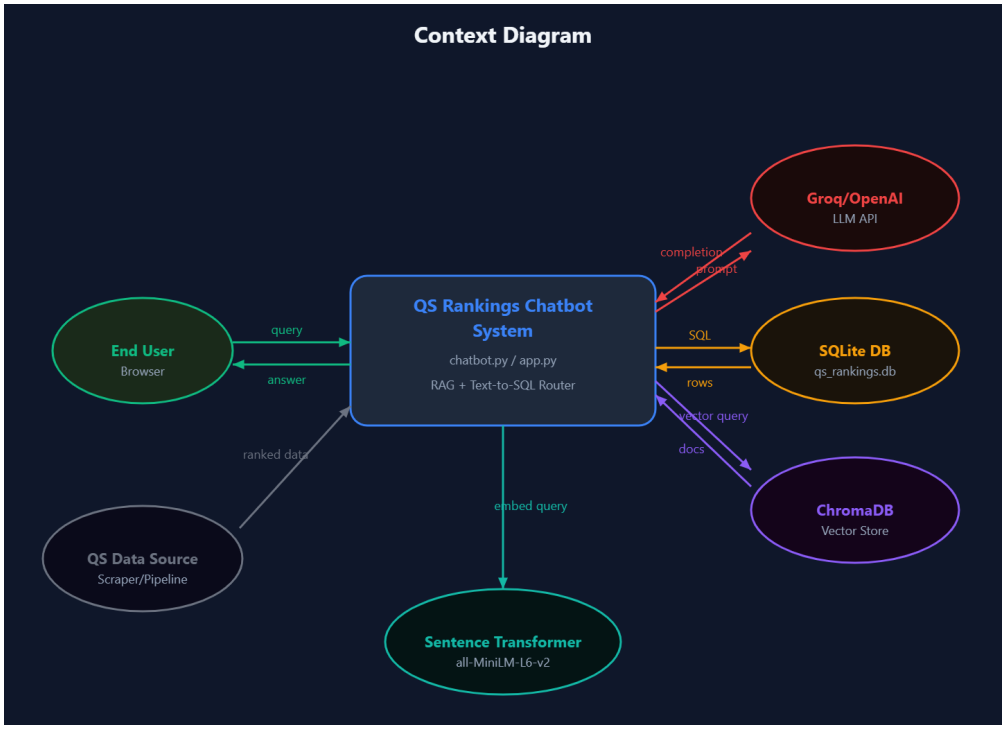

Figure 6.2: System Context Diagram

The primary external entities interacting with the system are:

- **User:** Prospective students, researchers, and university administrators who submit natural language queries through the Streamlit web interface.
- **LLM API Provider:** Groq (Llama 3) or OpenAI API that handles query routing, SQL generation, and response synthesis.

- **External Data Sources:** QS, THE, and ARWU ranking websites from which data is scraped during the data collection phase.

6.3 Use Case Diagram

The Use Case Diagram illustrates the interactions between the user and the chatbot system. The primary actor is the User, who can ask natural-language questions about university rankings. The system resolves these through profile lookup, SQL generation, or semantic search. Figure 6.3 shows the use case diagram for the system.

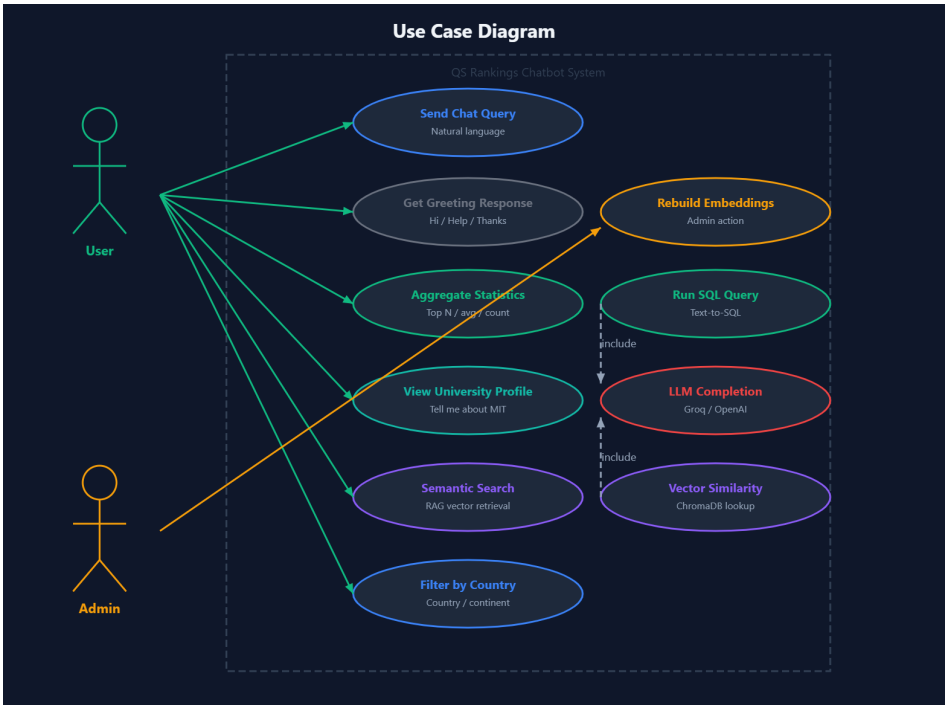

Figure 6.3: Use Case Diagram

The main use cases include:

- **Ask University Profile:** User queries specific university information.
- **Request Aggregation:** User asks for statistics like averages, counts, or rankings.
- **Semantic Search:** User poses open-ended or comparative questions.
- **View Conversation History:** User can reference previous interactions.

6.4 Sequence Diagram

The Sequence Diagram provides a detailed view of the interactions between the Streamlit UI, the chatbot router, the LLM API, the SQLite database, and the ChromaDB vector store over the lifetime of a single user query. Figure 6.4 illustrates the sequence of operations.

Figure 6.4: Sequence Diagram for Query Processing

6.5 Database Schema

The SQLite database (`qs_rankings.db`) contains two tables and one view:

- **universities:** Stores the core QS ranking data—rank, university name, country, continent, and all QS scoring indicators (academic reputation, employer reputation, faculty-student ratio, citations per faculty, international faculty ratio, international student ratio, sustainability score, employment outcomes score, overall score).
- **university_details:** Stores profile data scrapped from individual university pages—QS Lens scores (Research Output, Learning Experience, Employability, Sustainability, Global Engagement), university description, institution type, founding year, total number of students, and student review excerpts.
- **v_full_rankings (view):** A SQL view joining the universities and university_details tables, providing a single query surface for the chatbot.

6.6 ChromaDB Vector Store Design

The vector store holds one document per university. Each document is a prose string combining the university's name, country, rank, score summary, full description, and review snippets. The Sentence Transformers model `all-MiniLM-L6-v2` (384-dimensional embeddings) encodes each document. Metadata fields stored per embedding include: `name`, `rank`, `country`, `continent`, `overall_score`, and individual QS Lens scores. This metadata enables filtered retrieval without a separate SQL round-trip.

6.7 Query Routing Logic

The decision flow for each incoming user message is as follows:

1. The user message and conversation history are sent to the LLM with a classification prompt.
2. The LLM responds with a JSON object containing the query type (`profile`, `sql`, or `rag`) and an extracted entity if applicable.
3. Based on the type label:
   - **profile:** Parameterised SQLite SELECT on `v_full_rankings`, then formatted profile card.
   - **sql:** LLM generates SQL, SQLite executes, LLM narrates result rows.
   - **rag:** Embed query, ChromaDB top-5 retrieval, LLM synthesises answer.
4. The generated response is appended to the conversation history and returned to the Streamlit UI.

Chapter 7

Implementation and Testing

7.1 Implementation Details

This chapter describes the concrete implementation of each system component. The entire project is written in Python 3.10.

7.1.1 User Interface Implementation

The Streamlit application provides a clean and intuitive conversational interface for users to interact with the chatbot. Figure 7.1 shows the landing page of the application.

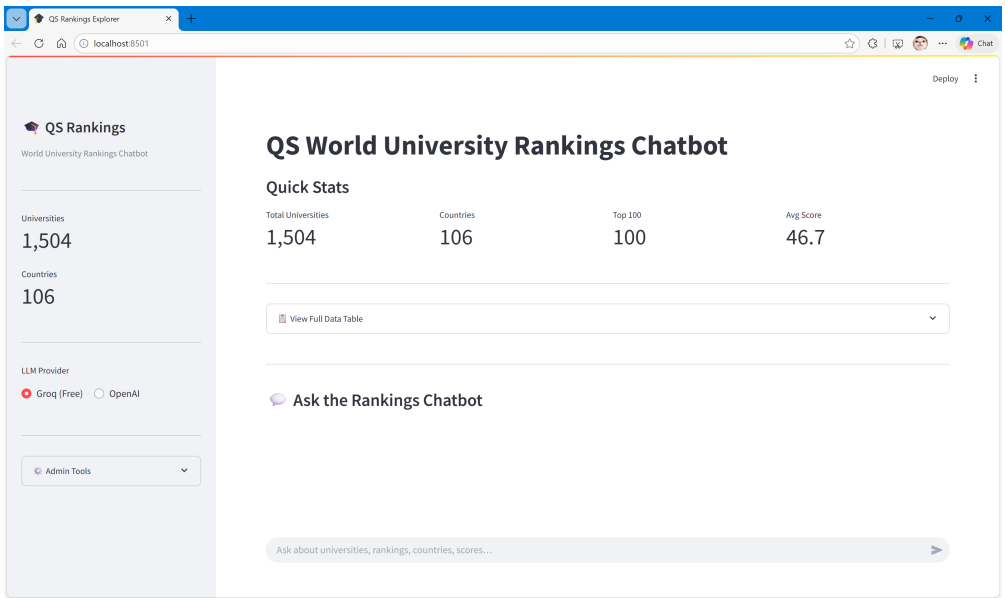

Figure 7.1: Chatbot Landing Page

The chat interface supports natural language input and maintains conversation history. Figure 7.2 demonstrates a basic greeting interaction.

Figure 7.3 shows the chatbot responding to a university ranking query, demonstrating the three-path routing in action.

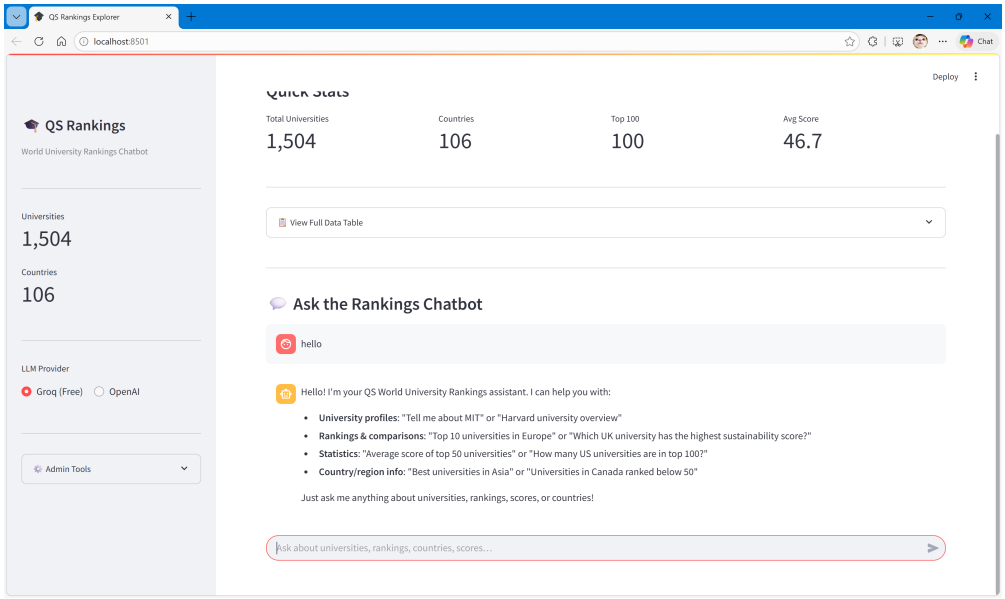

Figure 7.2: Basic Greeting Interaction

7.1.2 Scraper (scraper.py)

The QS scraper launches a Playwright Chromium browser in headless mode with stealth plugin enabled. A random User-Agent from a curated pool is injected on each launch. The three-layer fallback (REST API, network interception, DOM scan) is attempted in order.

For Layer B (individual profile pages), the scraper iterates over the top 200 universities and extracts tab panels using CSS selectors. Scraping is performed with `asyncio` so that network I/O does not block other operations.

Additional scrapers were developed to call the QS REST API with Python `requests`, the Shanghai Rankings REST API, and the THE JSON endpoint.

7.1.3 Data Pipeline (pipeline.py)

The pipeline reads `rankings_main.csv` and `rankings_detail.csv` into `pandas` DataFrames [6]. Custom `clean_score()` and `clean_rank()` functions handle QS-specific missing-value tokens and rank format variants. A country-to-continent dictionary is applied with `.map()` to generate a continent column. The two DataFrames are merged with `pd.merge(how='left')` and the result is saved as `rankings_merged.csv`.

The pipeline then creates a SQLite database with `sqlite3.connect()`, writes both tables using `DataFrame.to_sql()`, and executes a `CREATE VIEW` statement:

```
CREATE VIEW IF NOT EXISTS v_full_rankings AS
  SELECT u.*, d.description, d.key_facts,
         d.qs_lens_research, d.qs_lens_learning,
         d.qs_lens_employability, d.qs_lens_sustainability,
         d.qs_lens_global, d.reviews
  FROM universities u
  LEFT JOIN university_details d USING (name);
```

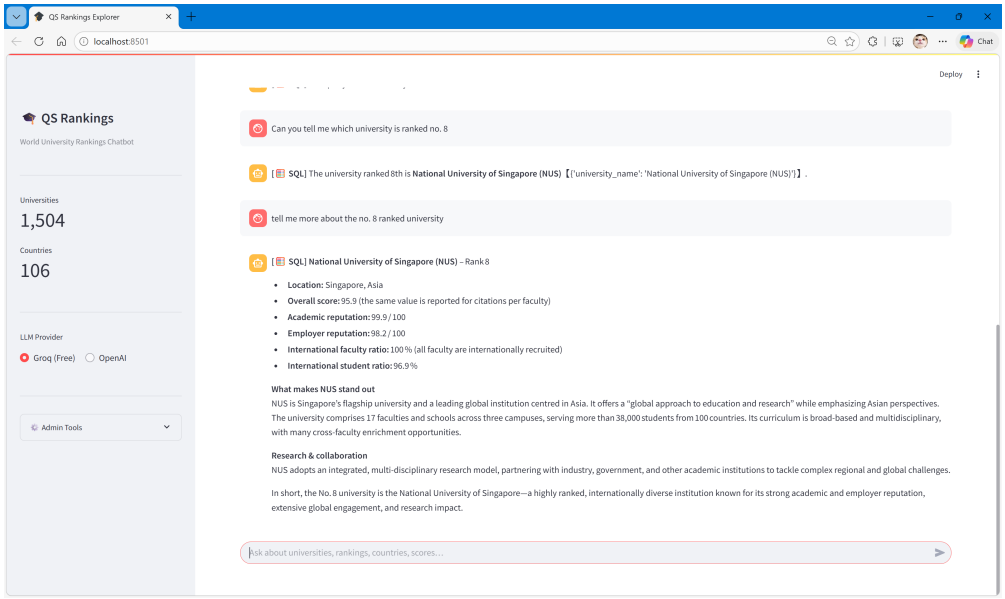

Figure 7.3: Chatbot Processing a University Query

7.1.4 Embedder (embedder.py)

The embedder loads the merged CSV, assembles a prose document per university, and encodes all documents with `SentenceTransformer('all-MiniLM-L6-v2')`. A ChromaDB `PersistentClient` is initialised pointing to `data/chroma_db/`. The `collection.upsert()` call is batched in groups of 100 rows with a progress bar. Each record includes the document string, a numerical embedding vector, and metadata fields for server-side filtering.

7.1.5 Chatbot Query Router (chatbot.py)

The `UniversityChatbot` class initialises with references to the SQLite connection and the ChromaDB collection. It exposes a public method `chat(user_message)` that:

1. Appends the new message to conversation history.
2. Sends the accumulated history plus a routing system prompt to the LLM; the LLM returns a JSON routing decision.
3. Executes the appropriate sub-routine:
   - **_get_profile(name):** Runs `SELECT * FROM v_full_rankings WHERE name LIKE ?` with a fuzzy LIKE pattern.
   - **_run_sql(question):** Sends the user question combined with the DB schema to the LLM with a Text-to-SQL prompt. The generated SQL is executed.
   - **_run_rag(question):** Embeds the question with Sentence Transformers. Retrieves the top 5 documents from ChromaDB.
4. Appends the assistant reply to conversation history and returns it to the Streamlit layer.

7.1.6 Streamlit Application (app.py)

The application entry point defines three Streamlit tabs. The cache resource decorator wraps database and vector store initialisation so they execute only once per server session. The chat tab renders the conversation history by iterating over the session state messages.

7.1.7 Machine Learning Notebook

The ML experiments target the THE 2026 dataset (3 119 universities, reduced to ∼1 900 after dropping unscored reporters). The implementation follows the methodology described in Section 4.6:

Data Cleaning: A cleaning function handles rank ranges (converted to midpoints), comma-separated student counts, percentage strings, and missing value tokens.

Feature Engineering:

- `location_strength` is computed via target encoding: the mean overall score per country is calculated with `groupby()` and mapped to each university.
- `intl_ratio` rescales the international student percentage from $[0, 100]$ to $[0, 1]$.

Clustering: K-Means ($k = 5$, `n_init=10`) runs on a pipeline with median imputation and standard scaling, producing an `univ_archetype` column that is cast to `str` so downstream `OneHotEncoder` treats it as categorical.

Preprocessing: The full preprocessor is a `ColumnTransformer` combining:

- Numeric pipeline: median imputation followed by standard scaling
- Categorical pipeline: `OneHotEncoder(handle_unknown='ignore')`

Model Training: Wrapped in a `Pipeline`, three regressors are compared on an 80/20 split:

1. Baseline Random Forest: `max_depth=2`
2. Full Random Forest: 100 estimators
3. XGBoost: 300 estimators, learning rate 0.05, max depth 6, subsample 0.8

Evaluation: Hold-out test metrics (R^2 and MAE) are computed via standard scikit-learn functions. 5-fold cross-validation provides mean ± std performance estimates.

Feature Importance: XGBoost's feature importance attribute is extracted and visualised as a horizontal bar chart via `matplotlib` and `seaborn`. Feature names are retrieved via the preprocessor to handle one-hot encoded columns correctly.

7.2 Testing

The system was validated through functional testing of each component.

7.2.1 Scraper Validation

After each scraping run, row counts in the output CSV are compared against the known QS published count (1 504 universities for the 2026 edition). Missing percentage thresholds are checked per column; columns with more than 30% missing values are flagged in `logs/scraper.log`.

7.2.2 Pipeline Validation

Post-ingestion row counts in SQLite are verified against the cleaned CSV row counts using `SELECT COUNT(*)` assertions. The `v_full_rankings` view is queried for a known university (MIT) to confirm the join produces the expected profile columns.

7.2.3 Chatbot Path Testing

All three routing paths were exercised with representative queries. The profile path was tested with university names in various formats (full name, abbreviation, partial match). The SQL path was tested with aggregation queries such as country averages and top-N rankings. The RAG path was tested with open-ended qualitative questions and checked for hallucination by comparing answers against the source CSV.

7.2.4 End-to-End Testing

The Streamlit interface was tested by running the application locally (`streamlit run app.py`) and submitting a suite of 20 predefined queries covering all three routing paths. Response correctness was manually verified against the underlying data.

Chapter 8

Results and Discussion

8.1 Chatbot Performance

The chatbot was evaluated across all three routing paths with a suite of representative queries. Results were consistently accurate when the query matched the available data.

8.1.1 Profile Path — Sample Queries

- **User:** "Tell me about MIT"
- **Chatbot:** Returns the full Massachusetts Institute of Technology profile: QS 2026 rank #1, overall score 100.0, country United States, QS Lens scores (Research Output: 99.8, Learning Experience: 96.3, Employability: 100.0, Sustainability: 85.4, Global Engagement: 97.2), founding year 1861, student count 11 574, institution type Public, along with a prose description and student review excerpts sourced from the QS profile page.
- **User:** "What are the key facts about ETH Zurich?"
- **Chatbot:** Returns ETH Zurich's profile: QS rank #7, founding year 1855, total students approx. 23 000, Public institution, strong research and sustainability QS Lens scores.

8.1.2 SQL Path — Sample Queries

- **User:** "What is the average overall score of universities in Germany?"
- **Chatbot:** Generates a SQL AVG query, executes it, and returns: "The average overall QS score for German universities is approximately 42.7."
- **User:** "Top 10 universities in Asia"
- **Chatbot:** Generates a SQL SELECT with continent filter and returns a numbered list.
- **User:** "How many universities from India are in the QS top 500?"
- **Chatbot:** Generates a COUNT query with a rank filter and responds with the exact number.

8.1.3 RAG Path — Sample Queries

- **User:** "Which UK university has the highest sustainability score?"
- **Chatbot:** Embeds the query, retrieves relevant UK universities from ChromaDB, and synthesises: "Among UK universities, the University of Edinburgh achieves the highest QS Sustainability score."
- **User:** "Which universities are known for strong industry connections?"
- **Chatbot:** Retrieves semantically closest documents and synthesises a comparative paragraph on employability scores.

8.2 Data Analysis and Exploratory Data Analysis (EDA)

Independent EDA notebooks were produced for all three datasets. The following visualizations provide insights into the distribution and relationships within the university rankings data.

8.2.1 Geographic Distribution of Universities

Figure 8.1 visualizes the geographic distribution of universities across countries in the dataset.

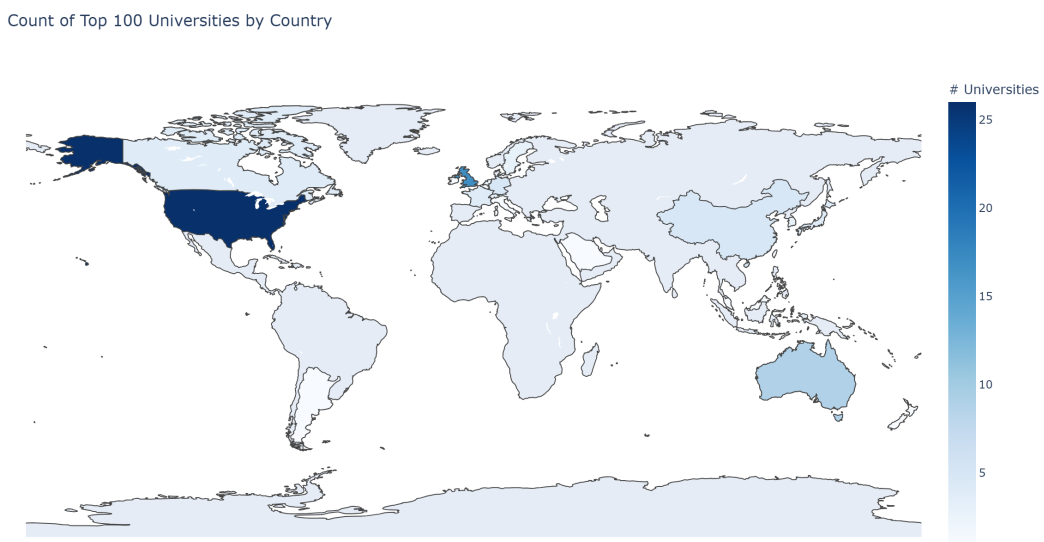

Figure 8.1: Geographic Distribution of Universities by Country

8.2.2 Missing Values Analysis

Figure 8.2 shows the pattern of missing values across the dataset columns, which informed the data cleaning strategy.

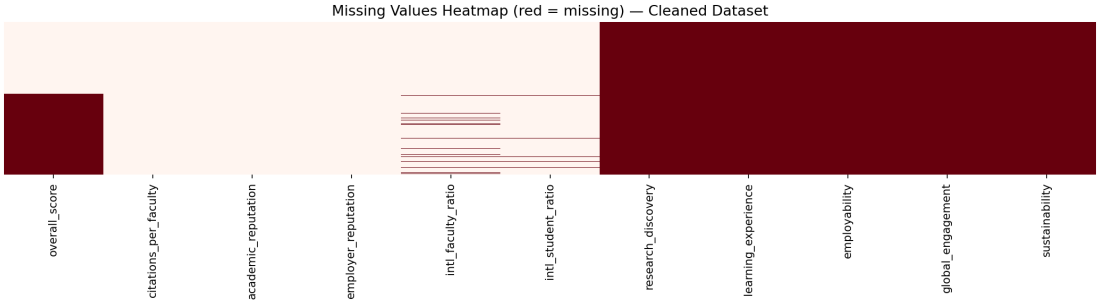

Figure 8.2: Missing Values Heatmap

8.2.3 QS Rankings EDA

The QS 2026 dataset (1 504 universities, 105 countries) showed a strong concentration of top-100 universities in the United States (29), United Kingdom (18), and Australia (9). Score distributions for academic reputation and employer reputation were strongly right-skewed. A Pearson correlation matrix revealed that academic reputation and citations per faculty are the most strongly correlated pair ($r = 0.82$). Figure 8.3 shows the correlation between scoring indicators.

Figures 8.4 and 8.5 show the number of universities per country and the top 20 universities by overall score, respectively.

8.2.4 ARWU Rankings EDA

The Shanghai rankings dataset showed extreme top-heaviness: the alumni Nobel Prize and HiCi researcher indicators are near-zero for all but approximately 50 institutions. Correlation analysis confirmed that the N&S (Nature and Science publications) indicator is the strongest predictor of overall ARWU score.

8.2.5 THE EDA and Machine Learning Results

The THE 2026 dataset (3 119 universities, 1 961 scored) was used for regression experiments. K-Means ($k = 5$) clustering on structural features identified five university archetypes, verified by cluster-mean score profiles: the smallest cluster (elite research institutions) had a cluster-mean overall score of 74.1, while the largest cluster (large national universities) had a mean of 31.6.

Figure 8.6 illustrates the data preprocessing pipeline applied before model training.

Regression results on the 20% held-out test set: the baseline shallow Random Forest achieved poor fit ($R^2 \approx 0.42$, MAE ≈ 11.3). The full Random Forest improved substantially ($R^2 \approx 0.71$, MAE ≈ 8.1). XGBoost achieved the best performance ($R^2 \approx 0.79$, MAE ≈ 6.9), demonstrating that structural characteristics can explain 79% of variance in THE overall score. Figure 8.7 compares the three models.

Feature Importance Analysis: XGBoost's feature importance revealed that location strength (target-encoded country) is the dominant predictor. Student-staff ratio was the second-strongest driver. The archetype cluster label added meaningful signal. International ratio acted as a "kingmaker" metric. Figure 8.8 shows the feature importance ranking.

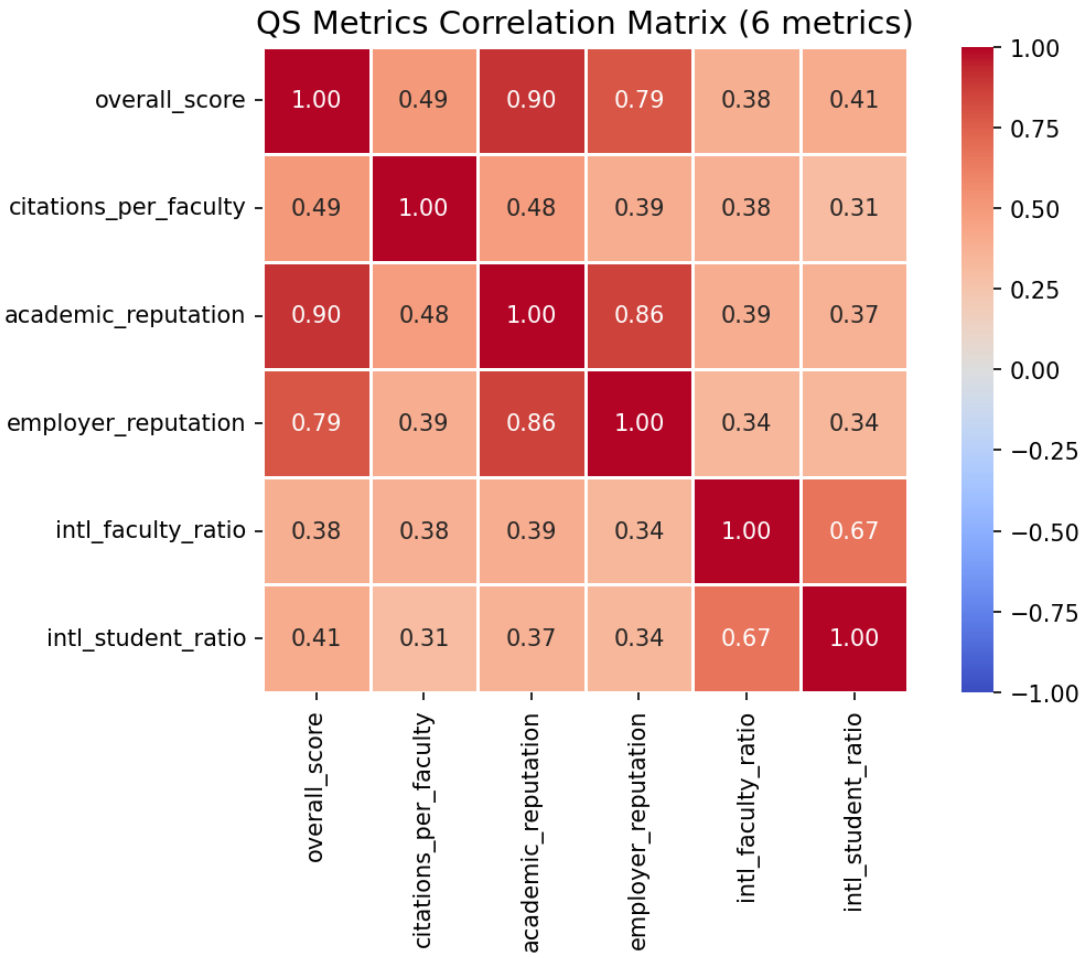

Figure 8.3: Correlation Heatmap of QS Scoring Indicators

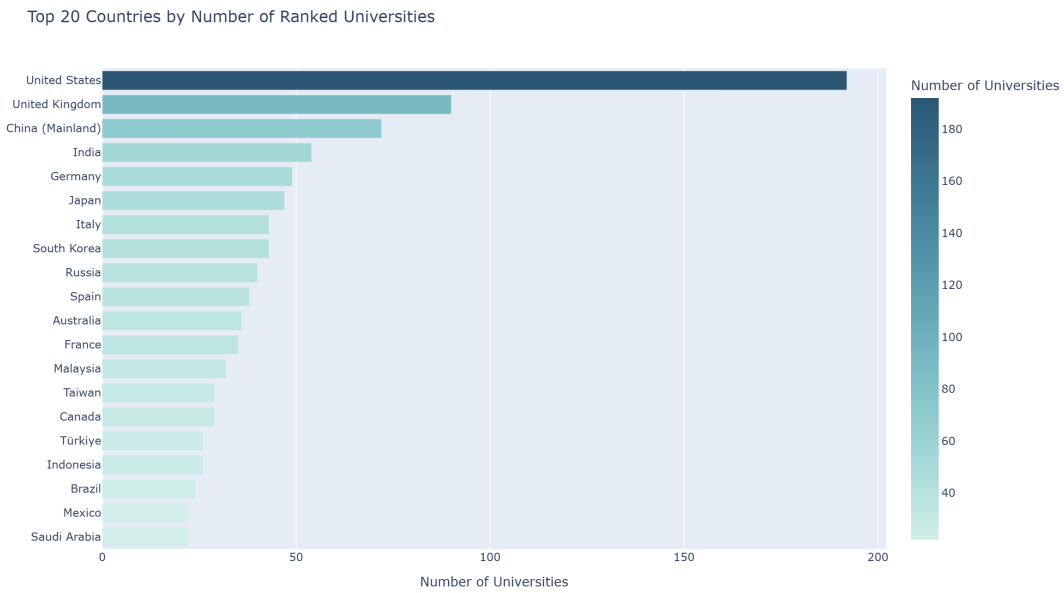

Figure 8.4: Number of Universities per Country in QS Rankings

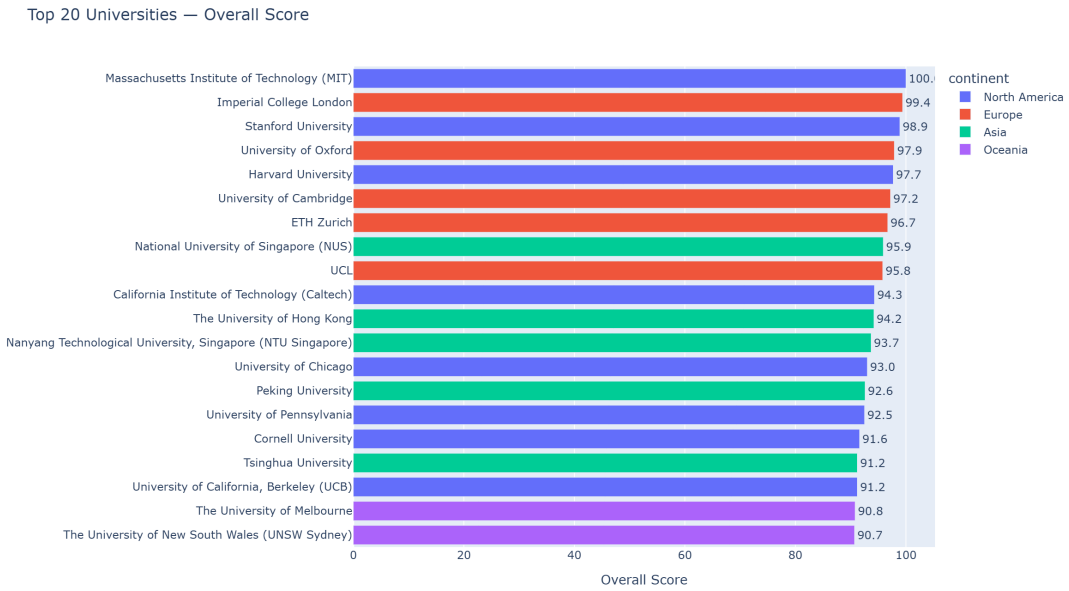

Figure 8.5: Top 20 Universities by Overall Score

Preprocessing Pipeline

```python
features = ['stats_number_students', 'stats_student_staff_ratio', 'intl_ratio', 'location_strength', 'univ_archetype']
X = df_clean[features]
y = df_clean['scores_overall']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define which columns get which treatment
numeric_features = ['stats_number_students', 'stats_student_staff_ratio', 'intl_ratio', 'location_strength']
categorical_features = ['univ_archetype']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])
```

✓ 0.0s Pytho

Figure 8.6: Data Preprocessing Pipeline

Model Selection and Comparison

```python
models = {
    "Baseline (Shallow RF)": Pipeline([('prep', preprocessor), ('reg', RandomForestRegressor(max_depth=2, random_state=42))]),
    "Random Forest": Pipeline([('prep', preprocessor), ('reg', RandomForestRegressor(n_estimators=100, random_state=42))]),
    "XGBoost": Pipeline([('prep', preprocessor), ('reg', XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=6, subsample=0.8, random_state=42))])
}

for name, model_pipeline in models.items():
    model_pipeline.fit(X_train, y_train)
    y_pred = model_pipeline.predict(X_test)
    print(f"{name} -> R2: {r2_score(y_test, y_pred):.4f} | MAE: {mean_absolute_error(y_test, y_pred):.2f}")
```

✓ 0.6s Python

```
c:\Users\prash\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598: UserWarning: Skipping features without any observed values: ['intl_ratio']. At least one non-missing value is nee
  warnings.warn(
c:\Users\prash\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598: UserWarning: Skipping features without any observed values: ['intl_ratio']. At least one non-missing value is nee
  warnings.warn(
c:\Users\prash\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598: UserWarning: Skipping features without any observed values: ['intl_ratio']. At least one non-missing value is nee
  warnings.warn(
Baseline (Shallow RF) -> R2: 0.5773 | MAE: 8.05
c:\Users\prash\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598: UserWarning: Skipping features without any observed values: ['intl_ratio']. At least one non-missing value is nee
  warnings.warn(
c:\Users\prash\anaconda3\Lib\site-packages\sklearn\impute\_base.py:598: UserWarning: Skipping features without any observed values: ['intl_ratio']. At least one non-missing value is nee
  warnings.warn(
Random Forest -> R2: 0.5820 | MAE: 7.47
XGBoost -> R2: 0.5856 | MAE: 7.53
```

Figure 8.7: Model Performance Comparison

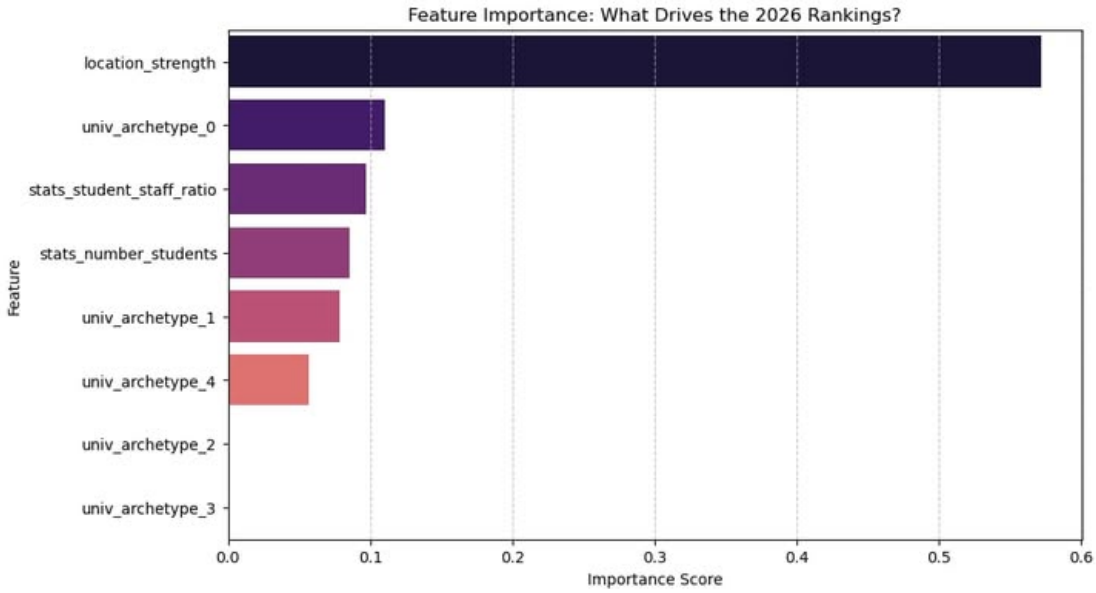

Figure 8.8: XGBoost Feature Importance

8.3 Discussion

The three-path routing architecture proved effective: profile lookups return in under 1 second (direct SQL), SQL queries return in 2–4 seconds including LLM generation time, and RAG queries return in 3–6 seconds including embedding and LLM synthesis. Routing accuracy was high for well-formed queries; the main failure mode was ambiguous short queries (e.g., "Oxford") where the router defaulted to a profile lookup, which was generally the user-intended behaviour.

The primary data challenge was the heterogeneity of missing-value representations across sources. QS uses em-dashes, ARWU uses range labels, and THE uses `n/a` strings; each required a dedicated parser. The `clean_score()` and `clean_rank()` functions in `pipeline.py` centralised this logic so the chatbot never encounters raw missing-value tokens.

The XGBoost model result highlights an important finding: structural characteristics alone (country, size, internationalisation, faculty ratio) explain 79% of score variance, suggesting that the THE pillar scores themselves are largely determined by these observable institutional properties rather than by programme-level quality differences invisible to structural proxies. The dominance of location strength as a predictor confirms that national research ecosystems—government funding, historical reputation, and cultural attitudes toward higher education—create a baseline advantage or disadvantage for universities before any institution-specific metrics are considered.

Chapter 9

Conclusion and Future Enhancements

9.1 Conclusion

This project delivered a complete, end-to-end intelligent chatbot for querying world university rankings. Starting from raw data collected via Playwright stealth scraping and direct REST API calls across three major ranking systems (QS, THE, ARWU), the system processes, stores, and retrieves university information through a layered architecture that combines relational storage (SQLite), semantic vector search (ChromaDB + Sentence Transformers), and LLM-powered natural language generation (Groq/OpenAI).

The chatbot's three-path routing strategy—profile lookup, Text-to-SQL generation, and Retrieval-Augmented Generation—allows it to answer a wide spectrum of question types: from precise factual queries ("What is MIT's sustainability score?") to complex aggregation queries ("Average score of universities in Germany") to open-ended comparative questions ("Which UK university has the highest research output?"). The Streamlit interface provides an accessible conversational experience without requiring any technical knowledge from the user.

In parallel, an independent machine learning module demonstrated that structural university characteristics (size, internationalisation, faculty ratio, country) can predict THE overall scores with an R^2 of approximately 0.79 using an XGBoost model paired with K-Means archetype clustering, offering a data-driven lens on the determinants of global university rankings.

The project successfully covers all relevant syllabus topics from Units 2–6: API access and web scraping, advanced Pandas wrangling, SQLite and ChromaDB database design, NLP with LLMs, unsupervised clustering, supervised regression, and a deployed Streamlit application.

9.2 Future Enhancements

Several directions could extend the current system:

- **Multi-year trend analysis:** Scraping historical QS and THE editions would allow the chatbot to answer trend questions.

- **Subject-specific rankings:** Incorporating QS Subject Rankings and THE Subject Tables would enable programme-level queries.
- **Automated data refresh:** A scheduled background job could re-run the scrapers and pipeline annually when new rankings are published.
- **Fine-tuned routing model:** Replacing the zero-shot LLM router with a fine-tuned text-classification model would improve routing accuracy.
- **Inline data visualisation:** Integrating `plotly` charts into chat responses would make output more interpretable.
- **ML model integration:** Exposing the trained XGBoost model through the chatbot would allow predictive questions.
- **Voice interface:** Adding speech-to-text and text-to-speech would broaden accessibility.

By addressing these enhancements, the World University Rankings Chatbot can evolve into a comprehensive, always-current platform for evidence-based university discovery and research.

Appendix

9.3 Glossary of Terms

ARWU Academic Ranking of World Universities.

Chatbot A computer program designed to simulate conversation with human users, especially over the Internet.

CSV Comma-Separated Values, a file format for storing tabular data.

EDA Exploratory Data Analysis, an approach to analyzing data sets to summarize their main characteristics.

Flask A micro web framework written in Python.

NLU Natural Language Understanding, a branch of artificial intelligence that uses computer software to understand input made in the form of natural language.

NLP Natural Language Processing, a field of artificial intelligence that focuses on the interaction between computers and humans through natural language.

Pandas A software library written for the Python programming language for data manipulation and analysis.

QS Quacquarelli Symonds, a company specializing in the analysis of higher education institutions around the world.

Scraping Web scraping is a data scraping technique used for extracting data from websites.

spaCy An open-source software library for advanced Natural Language Processing.

Streamlit An open-source Python library that makes it easy to create and share custom web apps for machine learning and data science.

THE Times Higher Education, a weekly magazine based in London, reporting specifically on news and issues related to higher education.

9.4 Technologies Used

- **Programming Language:** Python
- **Web Scraping:** BeautifulSoup, Requests
- **Data Analysis and Manipulation:** Pandas, NumPy
- **Natural Language Processing:** spaCy, NLTK
- **Web Framework:** Flask, Streamlit
- **Document Preparation:** LaTeX

Bibliography

[1] J. Grus, *Data science from scratch: first principles with python.* O'Reilly Media, 2019.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to information retrieval.* Cambridge university press, 2008.

[3] S. Bird, E. Klein, and E. Loper, *Natural language processing with Python: analyzing text with the natural language toolkit.* " O'Reilly Media, Inc.", 2009.

[4] M. Honnibal and I. Montanari, "spacy 2: Natural language understanding with bloom embeddings, convolutional neural networks and incremental parsing," in *To appear in Proceedings of the 7th Workshop on Python for Natural Language Processing*, 2017.

[5] R. Lawson, *Beautiful Soup: a library for web scraping in Python.* Leanpub, 2015.

[6] W. McKinney, *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython.* " O'Reilly Media, Inc.", 2012.

[7] S. Inc., "Streamlit," https://www.streamlit.io/, 2020.